

Informe Técnico - Parte 4 Docker

Cómo levantar el stack

```
cd parte4
sh setup.sh
docker compose up
```

setup.sh genera el par de claves ED25519 si no existe, copia la clave pública a cada runner y al manager, y copia la clave privada al directorio del manager para que el Dockerfile la incluya en la imagen. Tiene que correrse una vez antes del primer build.

El manager queda accesible en `http://localhost:8080`. Los runners no exponen puertos al host; solo son alcanzables desde dentro de `red_interna`.

Cómo conectarse por SSH a cada runner desde el manager

Desde dentro del contenedor manager:

```
ssh -i /home/manager/.ssh/id_ed25519 -o StrictHostKeyChecking=no
runner@runner-bash
ssh -i /home/manager/.ssh/id_ed25519 -o StrictHostKeyChecking=no
runner@runner-c
ssh -i /home/manager/.ssh/id_ed25519 -o StrictHostKeyChecking=no
runner@runner-ada
```

Los nombres de host son resueltos por Docker a través de `red_interna`. El usuario en todos los runners es `runner`.

Para entrar al manager desde el host:

```
docker exec -it manager /bin/bash
```

runner-bash

Imagen base

debian:bookworm-slim. Es la variante más reducida de Debian 12 que incluye un sistema base funcional. No trae compiladores, herramientas de debug ni gestores de paquetes adicionales más allá del mínimo. bash viene preinstalado; se agrega únicamente openssh-server.

Análisis de seguridad

La imagen base bookworm-slim tiene una superficie de ataque pequeña comparada con debian:bookworm. Aun así hereda dependencias como perl-base (requerido por openssh-server en Debian) y zlib1g, que tienen CVEs conocidos sin fix disponible en los repositorios de Debian 12. Ninguno de esos paquetes se invoca en el flujo de ejecución del runner, por lo que no son explotables directamente.

No hay compiladores ni intérpretes adicionales instalados, lo que elimina la posibilidad de compilar o ejecutar código arbitrario dentro del contenedor incluso si se obtuviera acceso SSH.

Configuración aplicada

Dockerfile:

```
FROM debian:bookworm-slim

RUN apt-get update && \
    apt-get install -y --no-install-recommends bash openssh-
    server && \
    rm -rf /var/lib/apt/lists/*

RUN useradd -m -s /bin/bash runner

RUN ssh-keygen -A && \
    mkdir -p /run/sshd && \
    echo "PermitRootLogin no" >> /etc/ssh/sshd_config && \
    echo "PasswordAuthentication no" >> /etc/ssh/sshd_config

RUN mkdir -p /home/runner/.ssh && chmod 700 /home/runner/.ssh
COPY id_ed25519.pub /home/runner/.ssh/authorized_keys
RUN chown -R runner:runner /home/runner/.ssh && \
    chmod 600 /home/runner/.ssh/authorized_keys

COPY app/ /home/runner/app/
RUN chmod +x /home/runner/app/paddock_manager_fixed.sh
```

EXPOSE 22

CMD `["/usr/sbin/sshd", "-D"]`

- Usuario no-root runner creado con useradd. sshd arranca como root (necesario para escuchar en el puerto 22 y manejar el cambio de identidad al conectar), pero la sesión SSH cae inmediatamente a runner.
- PermitRootLogin no y PasswordAuthentication no: solo acepta autenticación por clave pública, únicamente para el usuario runner.
- La clave pública del manager se copia como authorized_keys con permisos 600 y propietario runner:runner. El directorio .ssh tiene permisos 700.

docker-compose.yml:

- read_only: true: el sistema de archivos del contenedor se monta en solo lectura.
 - tmpfs: [/run/sshd, /tmp]: sshd necesita escribir un socket en /run/sshd al arrancar. Sin este tmpfs, el servidor falla con read_only: true. /tmp se agrega por si el script bash lo necesita.
 - cap_drop: ALL: se eliminan todas las capabilities de Linux. Este runner no necesita ninguna porque bash no realiza operaciones privilegiadas.
 - security_opt: no-new-privileges:true: impide que cualquier proceso hijo adquiera más privilegios que el proceso padre.
 - Sin ports: el runner no es accesible desde el host, solo desde red_interna.
-

runner-c

Imagen base

debian:bookworm-slim. La particularidad de este runner es que necesita compilar un binario C durante el build. Se usa multi-stage build para que el toolchain de compilación nunca llegue a la imagen final.

Análisis de seguridad

El problema de compilar dentro de la imagen final es que aunque se purguen gcc y libc-dev después de compilar, esos paquetes quedan registrados en las capas intermedias de la imagen. Un escáner de vulnerabilidades como Trivy analiza todas las capas, no solo el estado final del sistema de archivos, y reporta todas las vulnerabilidades del toolchain aunque los binarios ya no estén presentes. Esto infla el reporte significativamente (el toolchain de GCC trae consigo binutils, libasan, cpp, linux-libc-dev y sus dependencias, que suman cientos de CVEs).

La solución es multi-stage build: el stage de compilación nunca se incluye en la imagen final, por lo que las vulnerabilidades del toolchain no aparecen en el escaneo. La imagen final es equivalente en footprint a runner-bash.

Configuración aplicada

Dockerfile:

```
FROM debian:bookworm-slim AS builder

RUN apt-get update && \
    apt-get install -y --no-install-recommends gcc libc-dev && \
    rm -rf /var/lib/apt/lists/*

COPY app/restaurante.c /build/restaurante.c
RUN gcc -o /build/restaurante /build/restaurante.c -lpthread

FROM debian:bookworm-slim

RUN apt-get update && \
    apt-get install -y --no-install-recommends openssh-server && \
    rm -rf /var/lib/apt/lists/*

RUN useradd -m -s /bin/bash runner

RUN ssh-keygen -A && \
    mkdir -p /run/sshd && \
    echo "PermitRootLogin no" >> /etc/ssh/sshd_config && \
    echo "PasswordAuthentication no" >> /etc/ssh/sshd_config

RUN mkdir -p /home/runner/.ssh && chmod 700 /home/runner/.ssh
COPY id_ed25519.pub /home/runner/.ssh/authorized_keys
RUN chown -R runner:runner /home/runner/.ssh && \
    chmod 600 /home/runner/.ssh/authorized_keys

COPY --from=builder /build/restaurante /home/runner/app/
    restaurante
RUN chown runner:runner /home/runner/app/restaurante

EXPOSE 22
CMD ["/usr/sbin/sshd", "-D"]
```

- El stage builder instala gcc, compila el binario, y descarta todo lo demás.
- El stage final parte de debian:bookworm-slim limpio, instala solo openssh-server, y copia únicamente el binario compilado con COPY --from=builder.
- Configuración de usuario y SSH idéntica a runner-bash.

docker-compose.yml:

- `read_only: true, tmpfs: [/run/sshd, /tmp], security_opt: no-new-privileges:true`, sin ports: igual que `runner-bash`.
 - `cap_drop: ALL` más `cap_add: [SETUID, SETGID, NET_BIND_SERVICE]`: el binario C compilado con `pthread`s puede requerir `SETUID/SETGID` para operaciones internas de la `glibc` relacionadas con gestión de hilos. `NET_BIND_SERVICE` se mantiene por consistencia con `runner-ada`.
-

runner-ada

Imagen base

`debian:bookworm-slim`. Se instala `gnat` (GNU Ada Translator) para compilar el programa Ada. Después de compilar, `gnat` se purga con `apt-get purge` y `autoremove`.

Análisis de seguridad

A diferencia de `runner-c`, `runner-ada` no usa multi-stage build: purga `gnat` en la misma imagen después de compilar. Esto funciona correctamente porque `apt-get autoremove` después de `purge gnat` elimina todo el árbol de dependencias del compilador Ada (`gcc-12`, `gfortran`, librerías Ada y sus dependencias transitivas), dejando la imagen con el mismo footprint que `runner-bash`.

La diferencia con `gcc` es que el toolchain de GNAT tiene dependencias que se pueden remover limpiamente con `autoremove`, mientras que `gcc` deja residuos en capas anteriores que los escáneres detectan. Para un escáner que analice capas intermedias, `runner-ada` seguiría mostrando las vulnerabilidades de `gnat` en esas capas; si eso fuera un problema, la solución sería el mismo multi-stage que `runner-c`.

Configuración aplicada

Dockerfile:

```
FROM debian:bookworm-slim
```

```
RUN apt-get update && \  
    apt-get install -y --no-install-recommends gnat openssh-  
        server && \  
    rm -rf /var/lib/apt/lists/*
```

```
RUN useradd -m -s /bin/bash runner
```

```

RUN ssh-keygen -A && \
  mkdir -p /run/sshd && \
  echo "PermitRootLogin no" >> /etc/ssh/sshd_config && \
  echo "PasswordAuthentication no" >> /etc/ssh/sshd_config

RUN mkdir -p /home/runner/.ssh && chmod 700 /home/runner/.ssh
COPY id_ed25519.pub /home/runner/.ssh/authorized_keys
RUN chown -R runner:runner /home/runner/.ssh && \
  chmod 600 /home/runner/.ssh/authorized_keys

COPY app/ /home/runner/app/
RUN cd /home/runner/app && gnatmake juego.adb && chown
  runner:runner juego

RUN apt-get purge -y gnat && apt-get autoremove -y && rm -rf /var/
  lib/apt/lists/*

EXPOSE 22
CMD ["/usr/sbin/sshd", "-D"]

```

- gnatmake juego.adb compila el fuente Ada y genera el binario juego.
- La purga de gnat con autoremove elimina el compilador y todas sus dependencias del sistema de archivos activo.
- Configuración de usuario y SSH idéntica a los otros runners.

docker-compose.yml:

- read_only: true, tmpfs: [/run/sshd, /tmp], security_opt: no-new-privileges:true, sin ports: igual que runner-bash.
- cap_drop: ALL más cap_add: [SETUID, SETGID, NET_BIND_SERVICE]: mismo razonamiento que runner-c; el runtime Ada puede requerir operaciones de cambio de identidad de proceso.

manager

Imagen base

debian:bookworm-slim. Se instala openssh-client para conectarse a los runners, python3 para el servidor HTTP con soporte CGI, bash y procps. No corre sshd; solo actúa como cliente SSH y servidor web.

Análisis de seguridad

El manager tiene más paquetes que los runners (openssh-client, python3, procps) lo que implica más superficie de ataque. python3 arrastra libsqlite3-0 como dependencia, que tiene CVEs propios aunque no se use SQLite directamente.

La limitación de diseño más relevante es que la clave privada SSH se copia dentro de la imagen durante el build (COPY id_ed25519). Cualquier persona que acceda a la imagen puede extraerla. La alternativa correcta para producción sería montarla como Docker secret o como volumen en tiempo de ejecución; para este entorno de desarrollo la simplificación es aceptable.

StrictHostKeyChecking=no en las conexiones SSH del manager es justificable dentro de una red Docker controlada donde los hosts son conocidos y no cambian, pero no sería aceptable en una red no controlada.

Configuración aplicada

Dockerfile:

```
FROM debian:bookworm-slim

RUN apt-get update && \
    apt-get install -y --no-install-
    recommends openssh-client bash python3 procps && \
    rm -rf /var/lib/apt/lists/*

RUN useradd -m -s /bin/bash manager

RUN mkdir -p /home/manager/.ssh && chmod 700 /home/manager/.ssh
COPY id_ed25519 /home/manager/.ssh/id_ed25519
COPY id_ed25519.pub /home/manager/.ssh/id_ed25519.pub
RUN chown -R manager:manager /home/manager/.ssh && \
    chmod 600 /home/manager/.ssh/id_ed25519

COPY panel/ /var/www/panel/
RUN chown -R manager:manager /var/www/panel
COPY server.sh /home/manager/server.sh
RUN chmod +x /home/manager/server.sh

EXPOSE 8080
USER manager
CMD ["/bin/sh", "/home/manager/server.sh"]
```

- Usuario no-root manager. El contenedor arranca directamente como USER manager; el proceso principal nunca corre como root.
- La clave privada se copia con permisos 600 y propietario manager:manager.

- `/var/www/panel` se le da propiedad a `manager` antes de cambiar de usuario, porque `server.sh` necesita escribir el script CGI en ese directorio en runtime.

server.sh:

El script genera el CGI en `/var/www/panel/cgi-bin/estado` en runtime y levanta `python3 -m http.server --cgi 8080`. El CGI recibe el parámetro `host=` del query string, abre una conexión SSH al runner indicado, extrae CPU y memoria con `top` y `free`, y devuelve JSON. Si el runner no responde en 2 segundos devuelve `null`.

docker-compose.yml:

- Puerto 8080 expuesto al host para el panel web.
- `security_opt: no-new-privileges:true`.
- `depends_on: [runner-bash, runner-c, runner-ada]`: Docker espera que los runners estén levantados antes de iniciar el manager.
- Sin `read_only` ni `cap_drop`: Python necesita escribir el script CGI en runtime y el cliente SSH necesita capacidades básicas de red.